Practical Lab: Access Control Models

DAC, MAC, RBAC and ABAC

Mohamed Amine El Bacha

November 10, 2026

1 Introduction

Access control determines who can access a resource and which operations are permitted. Four models are studied: DAC (resource ownership), MAC (central policy), RBAC (user roles) and ABAC (attributes and conditions). The lab was performed on macOS, so Linux user/group commands were replaced by their macOS equivalents (`dseditgroup`, `dscl`) and SELinux was replaced by System Integrity Protection (SIP).

2 Discretionary Access Control (DAC)

The owner of a resource decides who can access it. Each file has an owner, a group, and read/write/execute bits for owner, group and others.

Guided experiment

```
touch report.txt
ls -l report.txt          # -rw-r--r-- user staff
chmod 740 report.txt
ls -l report.txt          # -rwxr----- user staff
```

The value `740` gives the owner read/write/execute, the group read only, and others nothing.

```
user@MacBook-Pro-de-USER-2 DAC_Lab % ls -l report.txt
-rw-r--r--  1 user  staff  0 10 nov.  11:16 report.txt
user@MacBook-Pro-de-USER-2 DAC_Lab % chmod 740 report.txt
```

Figure 1: Initial permissions and `chmod 740` on report.txt

```
user@MacBook-Pro-de-USER-2 DAC_Lab % ls -l report.txt
-rwxr-----  1 user  staff  0 10 nov.  11:16 report.txt
user@MacBook-Pro-de-USER-2 DAC_Lab % sudo groupadd accouting
```

Figure 2: Resulting `-rwxr---` permissions and group creation

Challenge 1 – Confidential Documents

```
mkdir -p company/finance
sudo dseditgroup -o create accounting
sudo chown root:accounting company/finance/secret.txt
sudo chmod 750 company/finance
sudo chmod 650 company/finance/secret.txt
ls -l company/finance    # -rw-r-x--- root accounting secret.txt
```

The directory gives the owner full access, the accounting group read/execute (traverse), and others nothing. The file is readable by the owner and the accounting group only.

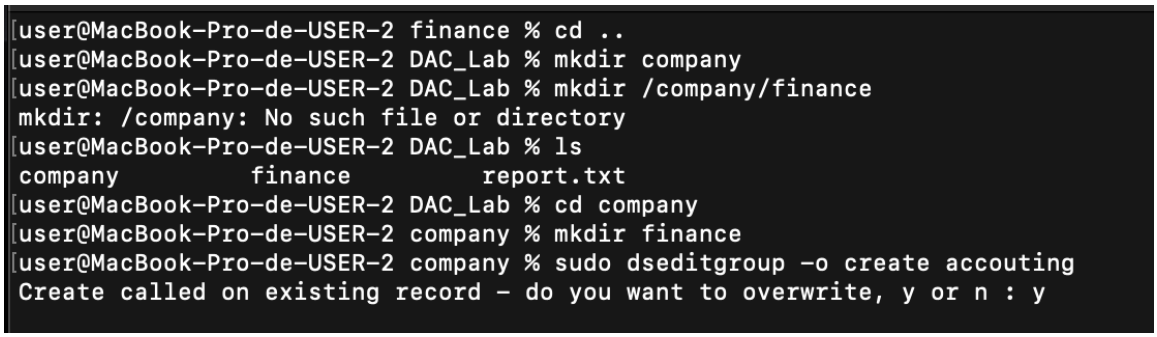


Figure 3: Creating the company and finance directories



Figure 4: Assigning ownership to the accounting group



Figure 5: Setting the document permissions

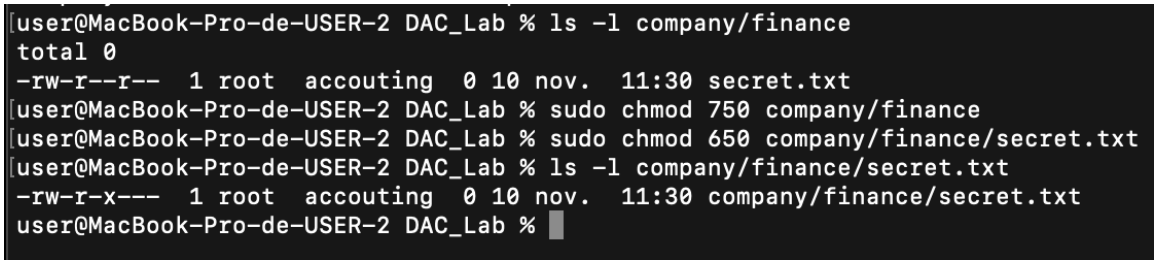


Figure 6: Final verification of the finance directory

Question 6. Directory `750`, file inside `600`. A member of the accounting group **cannot** read the file. The directory bits allow the group to traverse and list the directory, but the file's own `600` grants read to the owner only. Access requires permission on *both* the path and the file, so the group member is blocked at the file level.

3 Mandatory Access Control (MAC)

Access is enforced by a central security policy that resource owners cannot modify. On macOS, SELinux is not available; SIP is a comparable mandatory policy that even root cannot disable while booted.

```
csrutil status    # System Integrity Protection status: enabled
```

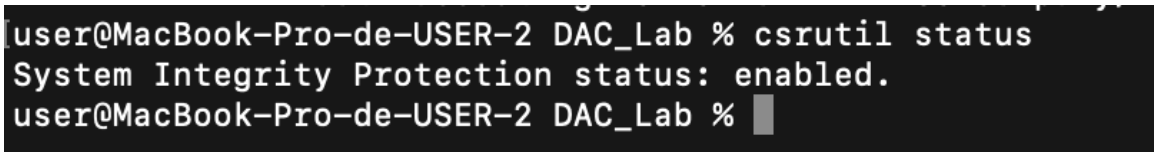


Figure 7: SIP status – a mandatory policy enforced centrally

Challenge 2 – Military Information System

Clearance order: Public < Confidential < Secret. The Bell–LaPadula rule *No Read Up* lets a user read only at or below their clearance.

	Doc_A (Secret)	Doc_B (Conf.)	Doc_C (Public)
Alice (Secret)	Yes	Yes	Yes
Bob (Confidential)	No	Yes	Yes
Charlie (Public)	No	No	Yes

1. See the matrix above.
2. Bob reading Document_A: **denied** (Confidential reading Secret is a read-up).
3. Alice writing to Document_C: **not allowed** (Secret writing to Public is a write-down).
4. *No Read Up* stops low-clearance users from seeing higher data; *No Write Down* stops high-clearance users from copying secret data into a lower level. Together they block any downward flow of information.
5. **Not necessarily.** MAC does not override DAC; both are enforced and access must pass both. If the owner's permissions deny access, the user is blocked even with sufficient clearance. DAC is set by the owner and can be changed by them; MAC is a central policy the owner cannot alter.

4 Role-Based Access Control (RBAC)

Permissions are assigned to organizational roles rather than to individual users. Linux groups (macOS: `dseditgroup` groups) simulate roles.

Guided experiment

```
# Create the roles
sudo dseditgroup -o create administrators
sudo dseditgroup -o create developers
```

```
sudo dseditgroup -o create viewers

# Create the users (macOS has no adduser)
for name in alice:701 bob:702 charlie:703; do
  user="${name%:*}"; uid="${name#*:}"
  sudo dscl . -create /Users/$user
  sudo dscl . -create /Users/$user UserShell /bin/bash
  sudo dscl . -create /Users/$user UniqueID $uid
  sudo dscl . -create /Users/$user PrimaryGroupID 20
  sudo dscl . -create /Users/$user NFSHomeDirectory /Users/$user
done

# Assign roles
sudo dseditgroup -o edit -a alice   -t user developers
sudo dseditgroup -o edit -a bob     -t user viewers
sudo dseditgroup -o edit -a charlie -t user administrators

groups alice   # ... developers ...
```

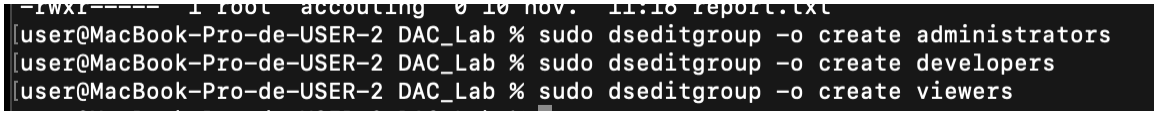


Figure 8: Creating the three role groups

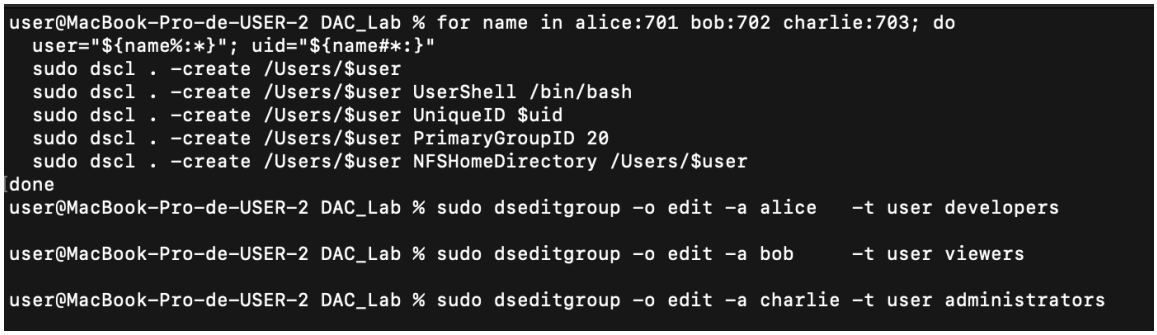


Figure 9: Creating the users and assigning them to role groups

Directories (note: `/company` is read-only on macOS, so `~/company` is used instead):

```
mkdir -p ~/company/source ~/company/reports
sudo chown root:developers ~/company/source  && sudo chmod 770 ~/company/source
sudo chown root:viewers    ~/company/reports && sudo chmod 750 ~/company/reports
ls -l ~/company
# drwxrwx--- root developers source
# drwxr-x--- root viewers    reports
```

```
chown: /company/source: No such file or directory
user@MacBook-Pro-de-USER-2 company % mkdir -p ~/company/source ~/company/reports
sudo chown root:developers ~/company/source  && sudo chmod 770 ~/company/source
sudo chown root:viewers    ~/company/reports && sudo chmod 750 ~/company/reports
ls -l ~/company
total 0
drwxr-x---  2 root  viewers     64 10 nov.  12:01 reports
drwxrwx---  2 root  developers  64 10 nov.  12:01 source
user@MacBook-Pro-de-USER-2 company %
```

Figure 10: Creating the role directories and setting base permissions

Challenge 3 – Enterprise Access Management

Required policy:

Role	Source code	Reports
Developer	Read/Write	Read
Viewer	No access	Read
Administrator	Full	Full

Does the basic configuration satisfy the policy? No. Standard POSIX permissions allow only one group per resource, so administrators get no access to either directory, and developers cannot read `reports`.

Correction with ACLs (macOS `chmod +a`), which allow several groups per resource:

```
sudo chmod +a "group:developers allow read,write,execute,delete" ~/company/source
sudo chmod +a "group:administrators allow read,write,execute,delete,add_file,
    add_subdirectory" ~/company/source

sudo chmod +a "group:viewers allow read,execute" ~/company/reports
sudo chmod +a "group:developers allow read,execute" ~/company/reports
sudo chmod +a "group:administrators allow read,write,execute,delete,add_file,
    add_subdirectory" ~/company/reports

ls -led ~/company/source ~/company/reports
```

Resulting ACLs confirm the full policy: on `source`, developers have read/write and administrators have full access; on `reports`, viewers and developers have read and administrators have full access.

```
user@MacBook-Pro-de-USER-2 company % ls -led ~/company/source ~/company/reports
drwxr-x---+ 2 root  viewers     64 10 nov.  12:01 /Users/user/company/reports
 0: group:administrators allow list,add_file,search,delete,add_subdirectory
 1: group:developers allow list,search
 2: group:viewers allow list,search
drwxrwx---+ 2 root  developers  64 10 nov.  12:01 /Users/user/company/source
 0: group:administrators allow list,add_file,search,delete,add_subdirectory
 1: group:developers allow list,add_file,search,delete
user@MacBook-Pro-de-USER-2 company %
```

Figure 11: ACLs applied – multiple groups per directory satisfy the policy

5 Attribute-Based Access Control (ABAC)

Authorization is decided from attributes of the subject, object, action and environment. Policy: department must match **and** clearance must be sufficient **and** time must be between 08:00 and 18:00.

Challenge 4 – Banking Access Policy

User	Decision	Failing attribute
Alice	GRANT	—
Bob	DENY	department (HR $\neq$ Finance)
Charlie	DENY	clearance (Public $<$ Secret)
David	DENY	time (22:00 outside window)

1. See the table above.
2. Failing attribute shown per row.
3. Alice requests an HR document: **denied** – her department (Finance) does not match HR.
4. David at 17:00: now inside the time window and all other attributes pass, so access is **granted**. The decision changes.
5. ABAC is more flexible than RBAC because it evaluates multiple dynamic attributes, including environment such as time, rather than static role membership. Fine-grained, context-aware rules can be expressed without creating a new role for every combination.

6 Conclusion

The four models were implemented and tested on macOS. DAC relies on owner-set permissions, MAC on a central policy the owner cannot override, RBAC on roles implemented through groups and ACLs, and ABAC on dynamic attribute evaluation for the most flexible, context-aware control.